
Module 13 — Reactive Programming and WebFlux

Lesson Notes

Spring Boot Edu



2026-09-24

Contents

1. Learning Goals	2
2. Concepts	2
2.1 Blocking and Non-Blocking	2
2.2 Mono, Flux and Subscription	2
2.3 Backpressure	3
3. Step-by-Step Examples	3
3.1 Operators	3
3.2 Errors and Retries	4
3.3 Time and Backpressure in Tests	5
3.4 Functional Endpoints and R2DBC	5
3.5 Reactive MongoDB and Mono.zip	7
3.6 Server-Sent Events	7
3.7 Reactive or Virtual Threads?	8
3.8 Testing	10
4. Common Mistakes and Best Practices	10
5. Summary	11
6. Further Reading	11

1. Learning Goals

By the end of this module you will be able to:

- Explain what “reactive” means: non-blocking, asynchronous streams with backpressure
- Work with Mono and Flux: create, transform, combine, handle errors and retry
- Test reactive code with StepVerifier, including virtual time
- Build WebFlux APIs with functional endpoints and with annotated controllers
- Access PostgreSQL with R2DBC and MongoDB reactively
- Push live data to browsers with Server-Sent Events
- Decide between reactive code and virtual threads

Prerequisites: Modules 03 (REST) and 07 (MongoDB) · **Estimated time:** 6 hours · **Docker required**

2. Concepts

2.1 Blocking and Non-Blocking

A classic Spring MVC application uses one thread per request. When the code waits for the database, the thread waits too. A **non-blocking** application instead registers “call me when the answer is there” and frees the thread. A few event-loop threads can then serve thousands of concurrent requests.

	Spring MVC (blocking)	Spring WebFlux (reactive)
Server	Tomcat, one thread per request	Netty, a few event-loop threads
Return types	Book, List<Book>	Mono<Book>, Flux<Book>
Database access	JDBC, JPA	R2DBC, reactive MongoDB driver
Code style	step by step	a pipeline of operators

! Important

On an event-loop thread you must **never block**: no `Thread.sleep`, no JDBC, no `block()`. One blocking call stalls every request on that thread.

2.2 Mono, Flux and Subscription

- A `Mono<T>` emits **0 or 1** element, then completes (or fails).

- A `Flux<T>` emits **0 to n** elements, then completes (or fails).
- Building a `Mono` or `Flux` does nothing. The work starts only when someone **subscribes**. In WebFlux, the framework subscribes to what your controller returns.

2.3 Backpressure

The subscriber tells the publisher how many elements it can handle (`request(n)`). A slow consumer is therefore not flooded by a fast producer. This contract comes from the Reactive Streams specification, which Reactor implements.

3. Step-by-Step Examples

With Docker running, start the application. PostgreSQL and MongoDB start from the root `compose.yaml`:

```
./mvnw -pl modules/13-reactive/lesson spring-boot:run
```

The data settings (under `spring:`):

```
# All modules share the "bookstore" database of compose.yaml; this one uses the schema
↪ "reactive"
# (R2DBC side: see book/R2dbcSchemaConfiguration).
flyway:
  schemas: reactive                # Flyway (JDBC) creates the tables before the app
  ↪ starts
mongodb:
  database: bookstore_reactive
```

3.1 Operators

```
public Flux<String> titlesInCapitals() {
    return Flux.just("Effective Java", "Spring in Action", "Java Puzzlers")
        .map(String::toUpperCase);           // runs only when
        ↪ someone subscribes
}

public Flux<Double> pricesOf(String... isbn) {
    return Flux.fromArray(isbn)
        .flatMap(this::lookupPrice);        // all lookups run at
        ↪ the same time
}
```

```

}

Mono<Double> lookupPrice(String isbn) {
    Double price = PRICES.get(isbn);
    return price == null
        ? Mono.error(new NoSuchElementException("No price for " + isbn))
        : Mono.just(price).delayElement(Duration.ofMillis(100)); // a slow remote
        ↪ call, without blocking
}

```

- map transforms each element synchronously.
- flatMap turns each element into a new publisher and runs them **concurrently**. The results arrive in the order they finish, not in the order of the input. The tour prints prices (in parallel): [95.0, 89.9], although 9780134685991 (89.90) was asked for first. Use concatMap or flatMapSequential when the order matters.
- delayElement waits without blocking a thread.

StepVerifier subscribes and checks every signal:

```

StepVerifier.create(basics.titlesInCapitals())
    .expectNext("EFFECTIVE JAVA", "SPRING IN ACTION", "JAVA PUZZLERS")
    .verifyComplete();

```

3.2 Errors and Retries

An error is a signal like an element. It travels down the pipeline and ends it, unless an operator handles it:

```

public Mono<Double> priceOrFallback(String isbn) {
    return lookupPrice(isbn)
        .onErrorResume(NoSuchElementException.class, e -> Mono.just(0.0)); //
        ↪ errors are signals too
}

public Mono<Double> flakyPriceWithRetry() {
    var attempts = new AtomicInteger();
    return Mono.defer(() -> attempts.incrementAndGet() < 3
        ? Mono.<Double>error(new IllegalStateException("timeout"))
        : Mono.just(89.90))
        .retryWhen(Retry.backoff(3, Duration.ofMillis(10))); // resubscribe: 10
        ↪ ms, 20 ms, ...
}

```

- onErrorResume replaces the error with another publisher, here a fallback price.
- retryWhen(Retry.backoff(...)) subscribes again after a growing pause. Mono.defer makes sure every attempt really calls the source again.

3.3 Time and Backpressure in Tests

A stream with one element per minute would make a test last minutes. With **virtual time**, StepVerifier moves a simulated clock:

```
StepVerifier.withVirtualTime(() -> basics.tick(Duration.ofMinutes(1)).take(3))
    .expectSubscription()
    .thenAwait(Duration.ofMinutes(3))           // no real waiting
    .expectNext(0L, 1L, 2L)
    .verifyComplete();
```

Backpressure can be tested by requesting elements one by one:

```
public Flux<Integer> numbers(int count) {
    return Flux.range(1, count)
        .doOnNext(n -> produced.incrementAndGet());           // counts what was
        ↪ really produced
}
```

ReactorBasicsTest requests 2, then 1, then cancels. Of `Flux.range(1, 1000)`, exactly 3 elements are produced.

3.4 Functional Endpoints and R2DBC

R2DBC is the non-blocking counterpart of JDBC. Spring Data offers the familiar repositories on top of it:

```
public interface BookRepository extends ReactiveCrudRepository<Book, Long> {

    Mono<Book> findByIsbn(String isbn);
}
```

WebFlux supports annotated controllers (as in Spring MVC), and **functional endpoints**, where routing is plain code:

```
@Configuration(proxyBeanMethods = false)
class BookRoutes {

    @Bean
    RouterFunction<ServerResponse> bookRouter(BookHandler books) {
        return route()
            .path("/api/books", builder -> builder
                .GET("", books::all)
                .GET("/{isbn}", books::one)
            )
    }
}
```

```

        .POST("", books::create))
    .build();
}
}

```

```

@Component
class BookHandler {

    private final BookRepository books;

    BookHandler(BookRepository books) {
        this.books = books;
    }

    Mono<ServerResponse> all(ServerRequest request) {
        return ServerResponse.ok().body(books.findAll(), Book.class); // streams
        ↪ the rows as they come
    }

    Mono<ServerResponse> one(ServerRequest request) {
        return books.findByIsbn(request.pathVariable("isbn"))
            .flatMap(book -> ServerResponse.ok().bodyValue(book))
            .switchIfEmpty(ServerResponse.notFound().build()); // empty
        ↪ Mono → 404
    }

    Mono<ServerResponse> create(ServerRequest request) {
        return request.bodyToMono(Book.class)
            .flatMap(books::save)
            .flatMap(saved -> ServerResponse.created(URI.create("/api/books/" +
                ↪ saved.isbn()).bodyValue(saved)));
    }
}

```

- `ServerResponse.ok().body(flux, Book.class)` writes the rows as they come from the database.
- `switchIfEmpty(...)` turns an empty Mono (no such book) into a 404.

i Note

Flyway has no R2DBC support, so it migrates the schema over JDBC before the reactive part starts. That is why the module also has the PostgreSQL JDBC driver. The R2DBC connections are pointed at the same schema (reactive) by a `ConnectionFactoryOptionsBuilderCustomizer` in `R2dbcSchemaConfiguration`.

3.5 Reactive MongoDB and Mono.zip

Reviews live in MongoDB, in a `ReactiveMongoRepository`. The details page needs the book from PostgreSQL **and** its reviews from MongoDB. `Mono.zip` subscribes to both at the same time and combines the results:

```
@GetMapping("/details")
Mono<ResponseEntity<BookDetails>> details(@PathVariable String isbn) {
    return Mono.zip(books.findByIsbn(isbn), // PostgreSQL ...
                   reviews.findByIsbnOrderByStarsDesc(isbn).collectList() // ... and
    ↪ MongoDB, at the same time
                   .map(both -> ResponseEntity.ok(toDetails(both.getT1(), both.getT2())))
                   .defaultIfEmpty(ResponseEntity.notFound().build()); // no book → zip
    ↪ is empty → 404
}
```

- The two queries run in parallel: the response time is that of the slower one, not the sum.
- If one of the Monos is empty (no such book), `zip` is empty too, and `defaultIfEmpty` turns that into a 404.

```
curl localhost:8080/api/books/9780134685991/details
{"isbn":"9780134685991","title":"Effective
↪ Java","price":89.90,"averageStars":0.0,"reviews":[]}
```

3.6 Server-Sent Events

With **Server-Sent Events (SSE)**, the server keeps the HTTP response open and writes an event whenever something happens. The browser's `EventSource` reconnects by itself.

```
@RestController
@RequestMapping("/api/orders")
class OrderStreamController {

    private final Sinks.Many<OrderEvent> orders =
    ↪ Sinks.many().multicast().directBestEffort(); // hot: no replay

    @PostMapping
    @ResponseStatus(HttpStatus.ACCEPTED)
    void place(@RequestBody OrderEvent order) {
        orders.tryEmitNext(order);
    }

    @GetMapping(path = "/stream", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
    Flux<ServerSentEvent<OrderEvent>> stream() {
```

```

var connected =
    ↪ ServerSentEvent.<OrderEvent>builder().comment("connected").build(); //
    ↪ sends the headers now
return orders.asFlux()
    .map(order -> ServerSentEvent.builder(order).event("order").build())
    .startWith(connected); // the connection stays
    ↪ open
}
}

```

- `Sinks.many().multicast()` is a **hot** publisher: every subscriber sees the events from the moment it subscribes. There is no replay of the past.
- WebFlux sends the response headers together with the first element. The `connected` comment makes sure the client gets the headers at once, and not only with the first order. Comments are ignored by clients.

```

curl -N localhost:8080/api/orders/stream
:connected

event:order
data:{"isbn":"9780134685991","quantity":2}

```

3.7 Reactive or Virtual Threads?

Java 21 brought **virtual threads**: threads so cheap that blocking is no longer expensive. Both approaches solve the same problem, waiting without wasting a scarce platform thread:

```

public Result reactive(int calls) {
    long start = System.nanoTime();
    Long count = Flux.range(1, calls)
        .flatMap(i -> Mono.just(i).delayElement(latency), calls) // waits
        ↪ without a thread
        .count()
        .block(); // only here: to
        ↪ measure
    return new Result(count == null ? 0 : count.intValue(),
        ↪ Duration.ofNanos(System.nanoTime() - start));
}

```

```

public Result virtualThreads(int calls) {
    long start = System.nanoTime();
    int results = 0;
    try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {

```

```

List<Future<Integer>> futures = new ArrayList<>();
for (int i = 1; i <= calls; i++) {
    int call = i;
    futures.add(executor.submit(() -> {
        Thread.sleep(latency);           // plain
        ↪ blocking code
        return call;
    }));
}
for (Future<Integer> future : futures) {
    future.get();
    results++;
}
} catch (InterruptedException | ExecutionException e) {
    throw new IllegalStateException(e);
}
return new Result(results, Duration.ofNanos(System.nanoTime() - start));
}

```

== 3.7 Reactive vs. virtual threads (200 calls of 200 ms)

reactive: 206 ms

virtual threads: 215 ms

Both take about as long as **one** call. Sequentially, it would be 40 seconds.

Choose virtual threads when ...	Choose reactive when ...
the application is a typical request/response service	you process streams: SSE, WebSocket, Kafka, backpressure
you use JDBC/JPA and blocking libraries	the whole chain is non-blocking (R2DBC, WebClient)
the team wants simple, step-by-step code	you need operators such as zip, timeout, retry across many sources

★ Tip

For most new Spring MVC applications, `spring.threads.virtual.enabled=true` gives most of the scalability with none of the new programming model. Reactive shines where data **flows**.

3.8 Testing

```
@TestConfiguration(proxyBeanMethods = false)
public class TestcontainersConfiguration {

    static final PostgreSQLContainer POSTGRES = new PostgreSQLContainer(
        DockerImageName.parse("pgvector/pgvector:0.8.6-
↪ pg18").asCompatibleSubstituteFor("postgres"));

    static final MongoDBContainer MONGO = new MongoDBContainer("mongo:8.0.32");

    @Bean
    @ServiceConnection // provides R2DBC *and* JDBC (Flyway) connection
    ↪ details
    PostgreSQLContainer postgresContainer() {
        return POSTGRES;
    }

    @Bean
    @ServiceConnection
    MongoDBContainer mongoContainer() {
        return MONGO;
    }
}
```

- WebTestClient tests WebFlux endpoints, and can read an SSE stream as a Flux (OrderStreamTest).
- @DataMongoTest also works for reactive repositories.
- For the R2DBC connection details from a PostgreSQL container, Boot needs org.testcontainers:testcontainers-r2dbc on the test classpath.

4. Common Mistakes and Best Practices

!! Caution

Never call `block()`, `Thread.sleep` or a JDBC method inside a reactive pipeline that runs on an event-loop thread. It freezes the server for everybody.

- **Do:** return `Mono/Flux` from controllers and let WebFlux subscribe.
- **Don't:** forget that nothing happens without a subscription. A `save(...)` whose result nobody subscribes to saves nothing.
- **Do:** use `flatMap` for concurrency and `concatMap` when the order matters.
- **Don't:** swallow errors silently. Every `onErrorResume` should be a conscious decision.
- **Do:** test with `StepVerifier` and virtual time instead of real sleeps.

- **Don't:** mix blocking libraries into a WebFlux application. If you must, move them to `Schedulers.boundedElastic()`.

5. Summary

- Reactive code is non-blocking: few threads serve many requests, and nothing may block them.
- Mono has 0-1, Flux 0-n elements. Work starts only on subscription.
- Operators transform (`map`), combine (`flatMap`, `zip`), handle errors (`onErrorResume`, `retryWhen`) and control time (`timeout`, `delayElement`).
- `StepVerifier` tests streams step by step, with virtual time and backpressure.
- WebFlux offers functional endpoints and annotated controllers. R2DBC and reactive MongoDB keep the database access non-blocking.
- SSE pushes live events over a normal HTTP response.
- Virtual threads are the simpler choice for request/response services. Reactive fits streams and complex asynchronous composition.

Next step: [Exercises](#)

6. Further Reading

- [Spring Framework — Web on Reactive Stack · Functional Endpoints](#)
- [Reactor Reference Guide · Which operator do I need?](#)
- [Spring Data R2DBC](#)
- [Spring Boot — Reactive Web Applications](#)
- [MDN — Server-Sent Events](#)
- [JEP 444 — Virtual Threads](#)